PRACTICAL LAB REPORT

Network Security and Protocol Analysis Laboratory

Network Protocol Analysis and Deep Packet Inspection using Wireshark

Practical Lab Report - IV

Submitted by	Mayank Raj
Programme	Master of Computer Applications (MCA)
Subject	Network Security Laboratory
Experiment No.	04 - Packet Capture and Protocol Dissection
Tool Used	Wireshark 4.x on Kali Linux (interface wlan0)
Date of Submission	20 September 2026

DECLARATION

I hereby declare that the packet captures, protocol dissections and analyses presented in this report were performed by me on a self-owned laboratory host and a controlled test network. All traffic was generated by me for academic purposes only. No third-party system was scanned, intercepted or accessed without authorisation.

Fourth in the series: Nmap Scanning | Nessus Assessment | Snort IDS | Wireshark Analysis

Table of Contents

Table of Contents 2

1. Introduction and Laboratory Objectives 4

1.1 Specific Objectives . . . 4

1.2 Laboratory Environment . . . 4

2. Packet Capture Methodology 6

2.1 Capture Filters versus Display Filters . . . 6

2.2 The Live Capture . . . 6

2.3 The Three Panes . . . 7

2.4 Traffic Generation . . . 7

3. TCP Three-Way Handshake Analysis 9

3.1 Isolating Connection Initiations . . . 9

3.2 The Complete Exchange . . . 9

3.3 Protocol Layering Demonstrated . . . 10

3.4 Sequence and Acknowledgement Semantics . . . 10

3.5 Retransmission Behaviour . . . 11

4. DNS Resolution Analysis 12

4.1 Transaction Structure . . . 12

4.2 Header Flags . . . 12

4.3 Name Encoding on the Wire . . . 12

4.4 Security Implications . . . 13

4.5 Correlation with Subsequent Traffic . . . 13

5. HTTP and HTTPS Comparative Analysis 14

5.1 HTTP Test Service . . . 14

5.2 HTTPS Request . . . 14

5.3 What Remains Visible . . . 15

5.4 Interpretation . . . 15

6. ICMP Analysis and Packet Loss Correlation 17

6.1 Tool Output . . . 17

6.2 Packet-Level Evidence . . . 17

6.3 Analysis . . . 18

6.4 Security Relevance 18

7. FTP Analysis and the Exposure Chain 19

7.1 Re-confirming the Nmap Finding 19

7.2 Service and Session Setup 19

7.3 The Cleartext Control Channel 20

7.4 Active and Passive Data Channels 21

7.5 The Exposure Chain 21

7.6 Remediation 21

8. Attack Traffic Analysis and Anomaly Indicators 22

8.1 Port Scan Signatures 22

8.2 Distinguishing Congestion from Attack 22

8.3 Other Indicators Relevant to This Capture 22

8.4 Analytical Workflow 23

9. Consolidated Findings 24

9.1 Recommendations 24

10. Conclusion 24

11. References 25

Appendix A: Display Filter Reference 26

Appendix B: List of Figures 26

1. Introduction and Laboratory Objectives

This report is the fourth in a continuing series of network security laboratory exercises. The earlier three reports established an exposure baseline for the laboratory host: an Nmap scan enumerated the reachable service surface, a Nessus assessment graded the weaknesses of those services, and a Snort deployment produced alerts when the corresponding traffic patterns appeared on the wire. Each of those tools, however, reports a conclusion rather than the evidence behind it. Wireshark closes that gap. It records every frame that crosses the monitored interface and allows the analyst to descend, field by field, from the Ethernet header down to the application payload.

The purpose of this experiment is therefore twofold. First, to demonstrate practical competence in live packet capture, display filtering and protocol dissection. Second, to use that competence to corroborate the findings of the previous three exercises - in particular to show, at the level of individual frames, why a cleartext service such as FTP constitutes a genuine exposure while a TLS-protected service does not.

1.1 Specific Objectives

- To perform a live capture on the wireless interface and interpret the packet list, packet details and packet bytes panes.
- To isolate and dissect a complete TCP three-way handshake and correlate it with the TLS session that follows it.
- To examine DNS resolution as a request-response transaction and map the returned records to subsequent connections.
- To contrast HTTP and HTTPS traffic in order to quantify precisely what an on-path observer can and cannot read.
- To analyse ICMP echo behaviour and evidence packet loss from the capture rather than from tool output alone.
- To capture an FTP control session and relate the cleartext control channel to the open port discovered by Nmap.
- To interpret retransmissions, duplicate acknowledgements and anomalous frames as indicators of network condition or malicious activity.

1.2 Laboratory Environment

Parameter	Value
Analysis host	Kali Linux (rolling), user `kali`
Capture interface	`wlan0` (promiscuous mode enabled)
Host IPv4 address	10.204.133.156/24
Host IPv6 address	2401:4900:a982:decf:f4e6:d6e3:3243:2550/64
Link-layer address	28:0c:50:ec:1d:0f (Intel)
Default resolver	10.204.133.125 (UDP/53)

Parameter	Value
Working directory	`~/wireshark-lab`
Capture file	`wireshark_wlan0SXALV3.pcapng`
Analyser	Wireshark 4.x GUI, default profile
Auxiliary tools	ping, nslookup, curl, python3 http.server, vsftpd, ftp, nmap 7.99

All traffic analysed in this report was generated deliberately by the author on the author's own host. The only remote endpoints contacted were public documentation and resolver addresses (example.com and 8.8.8.8), and the only service subjected to scanning and authentication was an FTP daemon installed on the loopback-reachable laboratory address itself.

2. Packet Capture Methodology

Wireshark was launched with sufficient privilege to place `wlan0` into promiscuous mode, and capture was started before any test traffic was generated so that the opening frames of every conversation would be present in the trace. Capture was allowed to run unfiltered; restriction of the view was done afterwards with display filters, which preserves the complete evidence set while still permitting focused analysis.

2.1 Capture Filters versus Display Filters

The distinction between the two filtering mechanisms is fundamental and is frequently confused. A capture filter uses Berkeley Packet Filter syntax and is applied by the kernel before frames are written to disk; anything it excludes is lost permanently. A display filter uses Wireshark's own syntax and merely hides frames in the interface, leaving the capture file intact. For laboratory work where the trace must support several different lines of enquiry, unfiltered capture with subsequent display filtering is the correct approach.

Property	Capture filter	Display filter
Syntax family	BPF, e.g. `tcp port 21`	Wireshark, e.g. `tcp.port == 21`
Applied at	Kernel, before writing	Application, after reading
Discarded data	Permanently lost	Retained in file
Typical use	Long or high-volume captures	Analysis of an existing trace
Used in this lab	None	`tcp.flags.syn == 1`, `dns`, `tls`, `icmp`, `tcp`

2.2 The Live Capture

The figure below shows the unfiltered capture in progress. The status bar records 228 frames captured with zero drops, which is the first quantity an analyst should verify: a non-zero drop count invalidates any conclusion drawn about loss or timing, since the missing frames may be an artefact of the capture host rather than of the network.

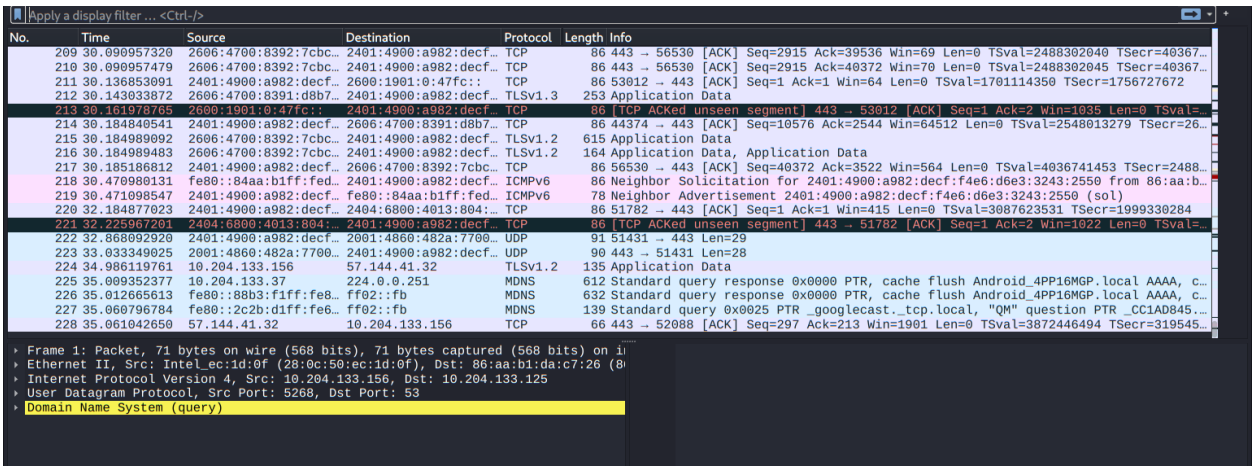


Figure 4.1: Live unfiltered capture on wlan0 showing the packet list and packet details panes; 228 frames captured, 0 dropped.

Several characteristics of a real wireless segment are visible even in this short excerpt. Frames 218 and 219 carry ICMPv6 Neighbour Solicitation and Neighbour Advertisement, the IPv6 equivalent of ARP address resolution. Frames 225 to 227 are multicast DNS announcements from other devices on the same subnet, including an Android handset advertising its services. Frame 1, selected in the details pane, is a DNS query carried over UDP source port 5268 to destination port 53 - the dissector has resolved the Ethernet, IPv4, UDP and DNS layers in sequence, which is the layered decomposition that makes Wireshark valuable.

2.3 The Three Panes

- **Packet list pane** - one row per frame with number, timestamp, addresses, the highest protocol the dissector could identify, frame length and a summary. Row colouring is rule-driven: black on red typically marks a TCP problem such as a retransmission, while pale blue marks ordinary traffic.
- **Packet details pane** - the selected frame expanded as a protocol tree. Each layer can be opened independently, which is how encapsulation is demonstrated.
- **Packet bytes pane** - the raw frame in hexadecimal with an ASCII gutter. Selecting a field in the tree highlights the exact bytes that encode it, proving the relationship between the dissected value and the wire format.

2.4 Traffic Generation

Because a passive capture of an idle host yields little of instructional value, traffic was generated deliberately from a second terminal. The sequence below produced the ICMP, DNS and HTTPS traffic examined in later sections.

```
$ ping -c 4 8.8.8.8
$ nslookup example.com
$ curl https://example.com
```

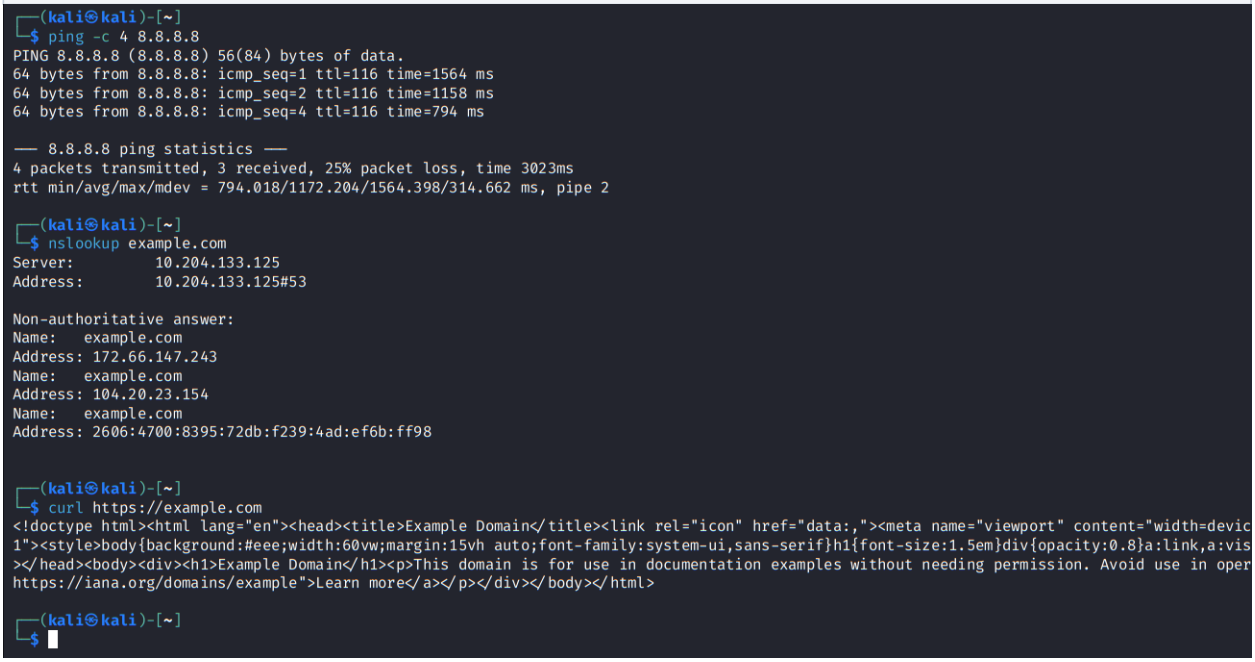


Figure 4.2: Terminal session used to generate ICMP, DNS and HTTPS traffic during the capture window.

The `nslookup` output is worth noting in advance of Section 4: the resolver at 10.204.133.125 returned two IPv4 addresses and one IPv6 address for a single name. This multi-record answer explains the dual-stack behaviour observed later, where the browser and `curl` preferred the IPv6 destination for the subsequent TLS connection.

3. TCP Three-Way Handshake Analysis

TCP is a connection-oriented protocol, and every connection begins with a three-frame exchange that synchronises sequence numbers in both directions and negotiates options. Isolating this exchange is the single most useful diagnostic skill in packet analysis, because the presence, absence or failure of the handshake distinguishes a working service from a filtered port, a closed port, or a scan.

3.1 Isolating Connection Initiations

The display filter `tcp.flags.syn == 1` was applied to reduce the trace to frames carrying the SYN flag. This matches both the initial SYN and the SYN-ACK, since the latter also has the SYN bit set, and so reveals every connection attempt in the capture together with the responses to those attempts.

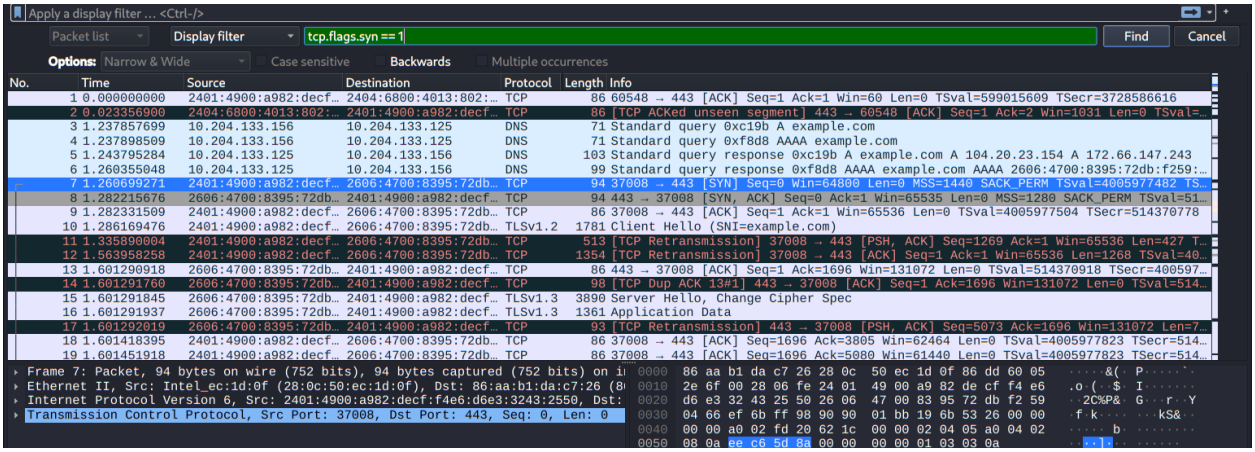


Figure 4.3: Display filter tcp.flags.syn == 1 applied; frame 7 selected, showing the initial SYN with Seq=0 and Len=0.

Frame 7 is the opening SYN from local port 37008 to destination port 443. The summary line reports `Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM`. The sequence number shown is relative - Wireshark substitutes zero for the true initial sequence number to make the trace readable, and the absolute value remains available in the tree. The zero payload length confirms that this frame carries control information only. The advertised maximum segment size of 1440 bytes reflects an IPv6 path with a standard 1500-byte MTU less 40 bytes of IPv6 header and 20 bytes of TCP header, and the SACK-permitted option announces that the client can acknowledge non-contiguous data, which becomes relevant when loss occurs.

3.2 The Complete Exchange

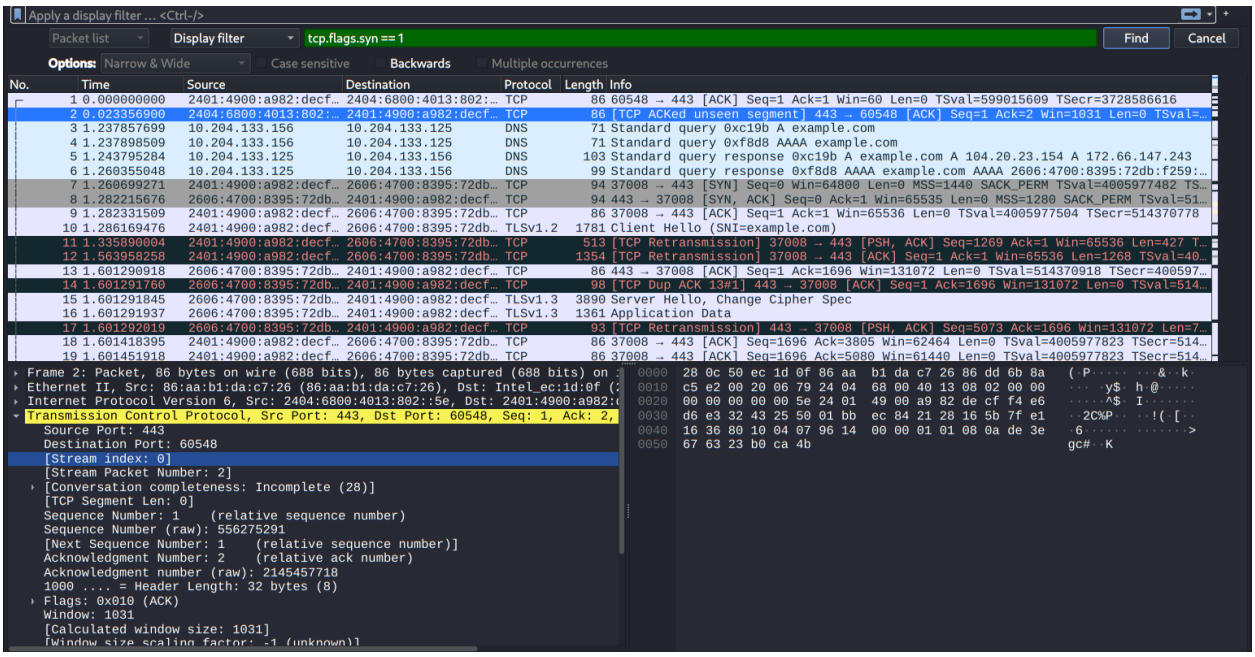


Figure 4.4: Frames 7, 8 and 9 forming the complete three-way handshake, followed by the TLS Client Hello in frame 10.

Frame	Time (s)	Direction	Flags	Significance
7	1.260699	Client → Server	SYN	Client proposes a connection; Seq=0, MSS=1440, SACK permitted.
8	1.282216	Server → Client	SYN, ACK	Server accepts, sends its own Seq=0 and Ack=1; MSS=1280.
9	1.282332	Client → Server	ACK	Client confirms; Seq=1, Ack=1. Connection is now established.
10	1.286169	Client → Server	PSH, ACK	TLSv1.2 Client Hello, SNI=example.com - first application data.

Two observations follow from the timings. The interval between frames 7 and 8 is approximately 21.5 milliseconds, which is a direct measurement of the round-trip time to the server and is more reliable than an ICMP-derived estimate because it uses the same transport and the same path as the traffic of interest. The interval between frames 9 and 10 is under 4 milliseconds, confirming that the TLS library began its handshake immediately upon receiving notification that the socket was connected.

3.3 Protocol Layering Demonstrated

Frame 10 is the point at which layering becomes explicit. The frame is dissected as Ethernet, then IPv6, then TCP, then TLS. The Client Hello it carries includes a Server Name Indication extension whose value, `example.com`, is transmitted in cleartext because the server must read it before a certificate can be selected and encryption established. This detail is revisited in Section 5, since it is the principal item of metadata that TLS does not conceal.

3.4 Sequence and Acknowledgement Semantics

The detail pane for frame 2 in the preceding figure exposes both the relative and the raw sequence numbers - relative 1 against raw 556275291 - alongside an acknowledgement number of 2 against raw 2145457718. The header length is reported as 32 bytes, that is, the 20-byte fixed header plus 12 bytes of options, which here carry the TCP timestamp values used for round-trip estimation and protection against wrapped sequence numbers. The window field of 1031 states how many further bytes the sender is prepared to accept; it is this field, changing over the life of a connection, that implements flow control.

3.5 Retransmission Behaviour

Filtering on `tcp` alone gives a fuller view of the same conversations and exposes their imperfections.

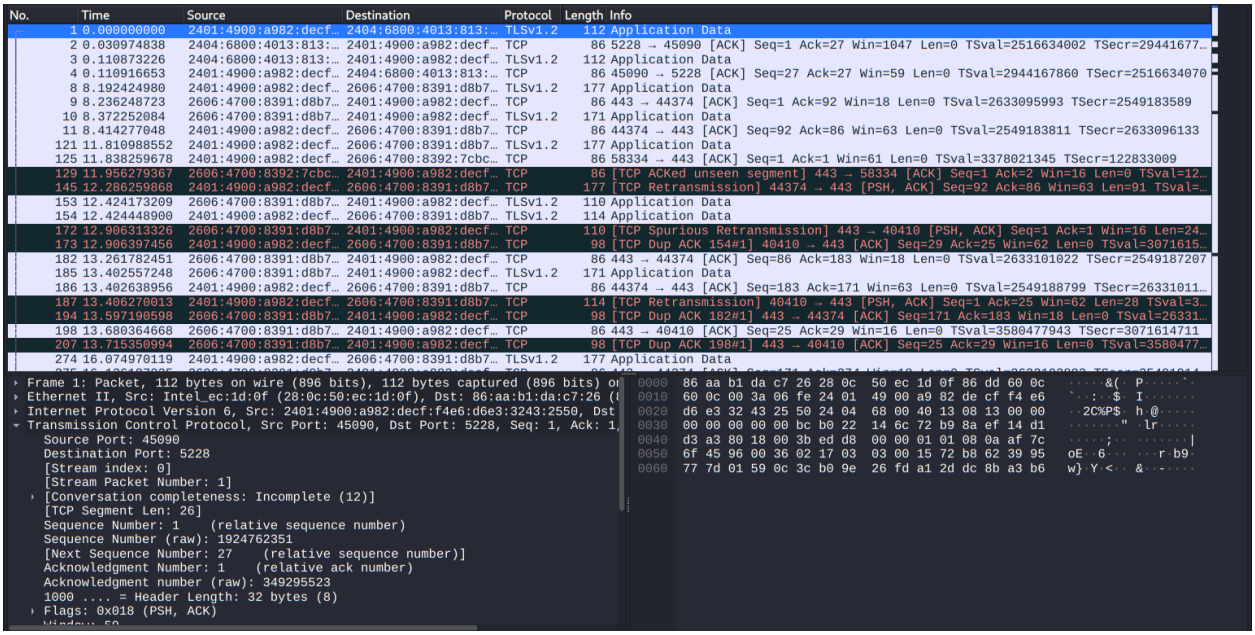


Figure 4.5: Broader TCP view: 121 of 398 frames displayed, showing retransmissions, spurious retransmissions and duplicate acknowledgements.

- **Retransmission** (frames 11, 12, 145, 187) - the sender's retransmission timer expired without an acknowledgement, so the segment was sent again.
- **Spurious retransmission** (frame 172) - the data was retransmitted although Wireshark had already observed it being acknowledged; the original acknowledgement was delayed or lost in transit rather than the data.
- **Duplicate ACK** (frames 173, 194, 207) - the receiver acknowledged the same sequence number again, signalling that a segment arrived out of order. Three such duplicates trigger fast retransmit without waiting for the timer.
- **ACKed unseen segment** (frames 2, 129) - an acknowledgement refers to data the capture never recorded, which is normal at the start of a trace or on a wireless link where some frames are missed by the capturing adapter.

Taken together these annotations describe a wireless path with intermittent loss rather than a malfunctioning host. The same signatures, however, appear during certain denial-of-service conditions, and Section 8 discusses how the two cases are distinguished.

4. DNS Resolution Analysis

Almost every user-initiated connection is preceded by a name resolution, and because DNS traditionally travels unencrypted over UDP it is both a rich source of analytical context and a significant privacy exposure. The filter `dns` reduced the capture to four frames constituting two complete transactions.

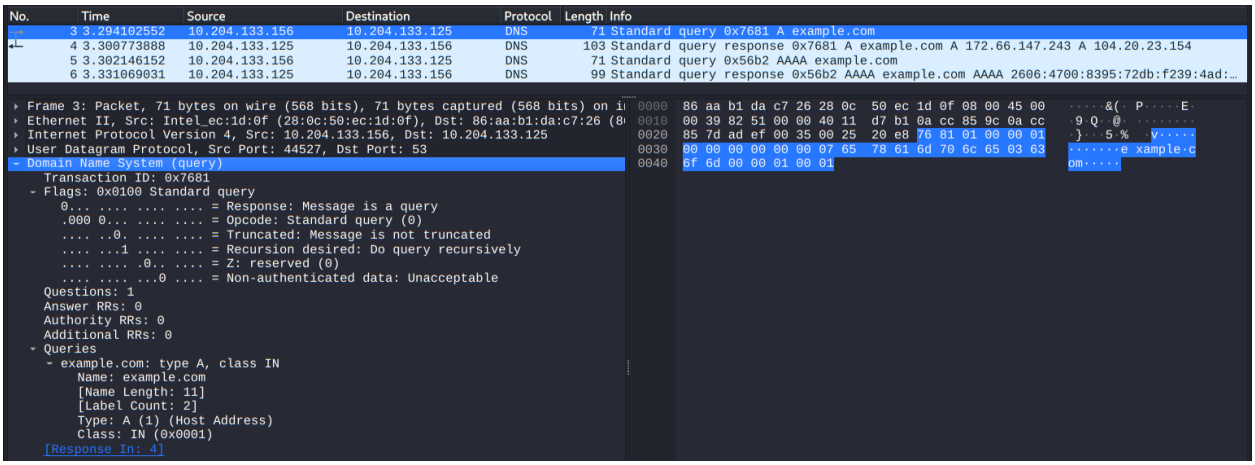


Figure 4.6: DNS query and response pairs for example.com; frame 3 expanded to show the query structure and the highlighted name encoding in the bytes pane.

4.1 Transaction Structure

Frame	Transaction ID	Type	Description
3	0x7681	Query	A record request for example.com, client port 44527 to resolver port 53.
4	0x7681	Response	Two A records returned: 172.66.147.243 and 104.20.23.154.
5	0x56b2	Query	AAAA record request for the same name.
6	0x56b2	Response	AAAA record 2606:4700:8395:72db:f239:4ad:ef6b:ff98 returned.

The transaction identifier is the mechanism by which a stateless UDP client matches a response to its query, and Wireshark uses the same field to generate the `[Response In: 4]` cross-reference in the detail pane. Two independent transactions were issued because the resolver library requested the IPv4 and IPv6 record types separately, which is the normal behaviour of a dual-stack host.

4.2 Header Flags

The expanded flags field of frame 3 reads 0x0100. Bit by bit this decodes as: response bit clear, so the message is a query; opcode zero, a standard query; the truncation bit clear, so the message fitted within a single UDP datagram; the recursion-desired bit set, instructing the resolver to pursue the answer on the client's behalf rather than returning a referral; and the non-authenticated data bit clear. The counts confirm one question and zero answer, authority and additional records, as expected for a query.

4.3 Name Encoding on the Wire

Selecting the query name in the detail pane highlights the corresponding bytes in the hexadecimal pane, where the ASCII gutter shows the readable string. DNS does not transmit the dot separator; each label is

preceded by a single length byte. The name is therefore encoded as 0x07 followed by `example`, then 0x03 followed by `com`, then a zero byte terminating the name. Wireshark reports a name length of 11 and a label count of 2, consistent with this encoding.

4.4 Security Implications

Because the query travels in plaintext, any party on the path - including the operator of the wireless segment - can enumerate every hostname the client resolves. This produces a complete browsing profile without any need to break encryption, and it remains true even when all subsequent traffic is protected by TLS. Section 5 shows the same name reappearing in the TLS Server Name Indication, so a passive observer has two independent opportunities to learn the destination. The practical mitigations are DNS over TLS or DNS over HTTPS, which move resolution inside an encrypted channel, and DNSSEC, which addresses the separate problem of response integrity. Neither was in use on this network, which is itself a finding worth recording.

4.5 Correlation with Subsequent Traffic

The address 2606:4700:8395:72db:f239:4ad:ef6b:ff98 returned in frame 6 reappears as the destination of frames 7 to 18 in Section 3. This chain - query, response, handshake, encrypted session - is the canonical illustration of why an analyst filters on DNS first when reconstructing a user action from a capture.

5. HTTP and HTTPS Comparative Analysis

The contrast between HTTP and HTTPS is the clearest demonstration available of what transport encryption achieves. Both were exercised: a local HTTP server on port 8080 and a TLS request to a public host.

5.1 HTTP Test Service

A minimal document root was created and served with Python's built-in HTTP server.

```
$ mkdir -p ~/wireshark-lab && cd ~/wireshark-lab
$ echo "Wireshark HTTP Test" > index.html
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```



Figure 4.7: Python HTTP test server bound to port 8080 in the laboratory working directory.

Requests to this service are dissected by Wireshark as HTTP, and every element of the exchange is legible: the request method and URI, the Host and User-Agent headers, the status line, the content type and length, and the entire response body. The 'Follow HTTP Stream' function reassembles the conversation into a readable transcript. Nothing about the transaction is concealed from an observer positioned anywhere along the path.

5.2 HTTPS Request

```
$ curl https://example.com
```

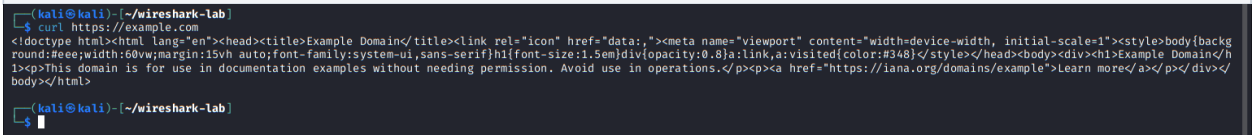


Figure 4.8: curl request to example.com over HTTPS, returning the document body to the terminal.

The document is rendered normally in the terminal, so from the user's perspective the two protocols are indistinguishable. On the wire they are not.

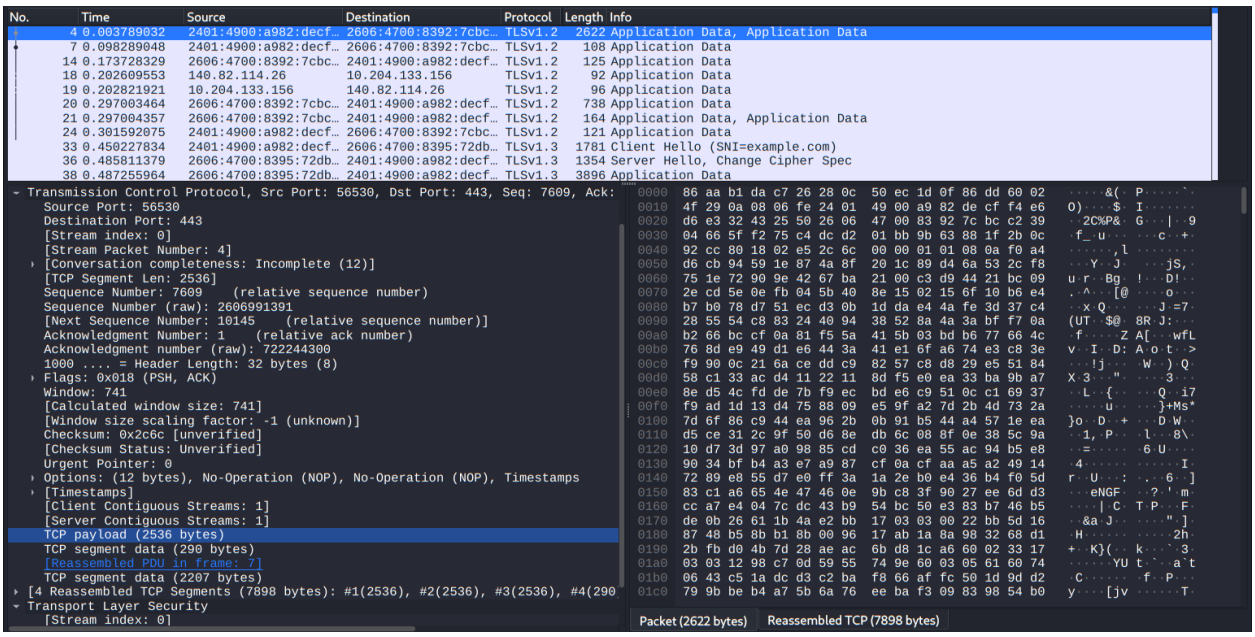


Figure 4.9: Display filter tls applied; 28 of 74 frames displayed, showing Client Hello, Server Hello and Application Data records with TCP segment reassembly.

5.3 What Remains Visible

The filtered view shows the handshake in frames 33 and 36 followed by Application Data records. The detail pane for the selected frame illustrates reassembly: four TCP segments of 2536, 2536, 2536 and 290 bytes are combined into a single 7898-byte protocol data unit, because TLS records routinely exceed the path MSS. Wireshark's reassembly makes the record boundaries comprehensible while the payload itself stays opaque - the hexadecimal pane shows high-entropy bytes with no recognisable structure in the ASCII gutter.

Observable	HTTP (port 8080)	HTTPS (port 443)
Destination IP and port	Visible	Visible
Hostname	Visible in Host header	Visible in TLS SNI extension
Request method and URI	Fully visible	Encrypted
Request and response headers	Fully visible	Encrypted
Cookies and session tokens	Fully visible	Encrypted
Response body	Fully visible	Encrypted
Status code	Visible	Encrypted
Certificate details	Not applicable	Visible in TLS 1.2; encrypted in TLS 1.3
Timing, volume and frame sizes	Visible	Visible

5.4 Interpretation

Encryption protects content and credentials; it does not protect metadata. An observer still learns which host was contacted, when, for how long and with how much data transferred, and traffic-analysis techniques can infer a surprising amount from those quantities alone. The correct conclusion for a security assessment is that TLS eliminates the credential-theft and content-tampering risks that dominate the HTTP case, while leaving a residual metadata exposure that must be addressed by other means if it matters. It is also worth noting that the capture contains TLS 1.2 and TLS 1.3 sessions concurrently, the latter encrypting the certificate exchange itself and so revealing strictly less than the former.

6. ICMP Analysis and Packet Loss Correlation

ICMP carries control and diagnostic messages for IP. The echo request and echo reply pair used by `ping` is the simplest possible reachability test, and precisely because it is simple it provides an excellent opportunity to demonstrate that tool output and packet evidence must be reconciled rather than assumed to agree.

6.1 Tool Output

```
┌──(kali㉿kali)-[~/wireshark-lab]
└─$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=1088 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=1051 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=1325 ms

── 8.8.8.8 ping statistics ──
4 packets transmitted, 3 received, 25% packet loss, time 3004ms
rtt min/avg/max/mdev = 1050.980/1154.432/1324.794/121.384 ms, pipe 2

┌──(kali㉿kali)-[~/wireshark-lab]
└─$
```

Figure 4.10: Output of ping -c 4 8.8.8.8 reporting three replies, 25 percent loss and round-trip times above one second.

Four requests were transmitted and three replies received, giving 25 percent loss. The round-trip times, between 1051 and 1325 milliseconds, are roughly fifty times what a well-provisioned path to a public anycast resolver should produce and indicate a congested or weak wireless link. The `pipe 2` annotation records that `ping` had two requests outstanding simultaneously, which occurs when the round-trip time exceeds the one-second send interval.

6.2 Packet-Level Evidence

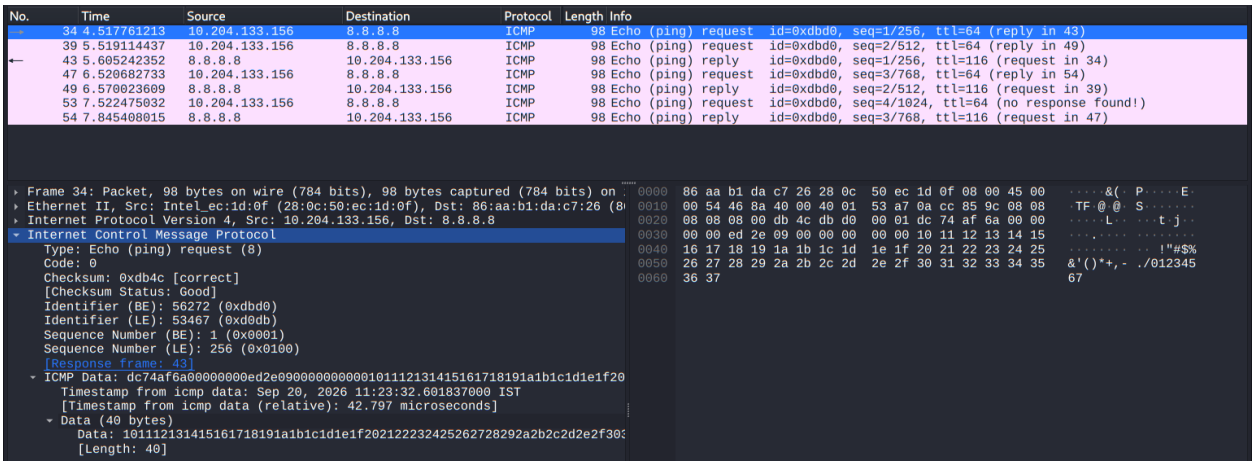


Figure 4.11: Display filter icmp applied; seven frames displayed with frame 34 expanded to show the ICMP header, identifier, sequence number and payload.

Frame	Time (s)	Type	Sequence	TTL	Outcome
34	4.517761	Echo request	1/256	64	Reply in frame 43

Frame	Time (s)	Type	Sequence	TTL	Outcome
39	5.519114	Echo request	2/512	64	Reply in frame 49
43	5.605242	Echo reply	1/256	116	Request in frame 34
47	6.520683	Echo request	3/768	64	Reply in frame 54
49	6.570024	Echo reply	2/512	116	Request in frame 39
53	7.522475	Echo request	4/1024	64	No response found
54	7.845408	Echo reply	3/768	116	Request in frame 47

6.3 Analysis

The capture confirms the tool's summary and adds detail the tool cannot supply. Frame 53 carries sequence 4 and is annotated `no response found`, so the loss is localised to a specific request rather than being a statistical abstraction. Equally important, the request was transmitted successfully by the local host: the loss occurred beyond the capture point, either on the wireless segment or upstream, and is not attributable to the host's own network stack.

- **Sequence number representation** - Wireshark displays both interpretations, for example 1/256, because the two-byte field is read as big-endian 1 and little-endian 256. The same duality appears in the identifier, shown as 56272 and 53467. This is a display convention, not two distinct fields.
- **Time-to-live asymmetry** - requests leave with TTL 64 while replies arrive with TTL 116. Assuming the remote host also used an initial value of 128, twelve router hops were traversed on the return path.
- **Payload** - the 40-byte data section is an incrementing byte pattern preceded by a timestamp inserted by ping. The timestamp is how round-trip time is calculated, and Wireshark reads it back to report the relative offset.
- **Checksum** - reported as 0xdb4c and validated as correct, confirming the frame was not corrupted between transmission and capture.

6.4 Security Relevance

The ICMP echo facility is frequently abused. Host discovery sweeps generate large volumes of echo requests from a single source, smurf-style amplification uses spoofed source addresses with broadcast destinations, and covert channels hide data in the payload field - which is why the payload of frame 34 was inspected and found to contain the expected benign incrementing pattern rather than arbitrary data. Many hardened networks therefore rate-limit or suppress echo replies entirely, at the cost of losing a genuinely useful diagnostic.

7. FTP Analysis and the Exposure Chain

This section deliberately closes the loop with the earlier reports in the series. The Nmap exercise identified an open FTP port; the Nessus exercise classified cleartext authentication as a weakness; this section supplies the packet-level proof.

7.1 Re-confirming the Nmap Finding

```
$ sudo nmap -sS 10.204.133.156
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-20 11:31 +0530
Nmap scan report for 10.204.133.156
Host is up (0.000041s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT STATE SERVICE
21/tcp open ftp
```

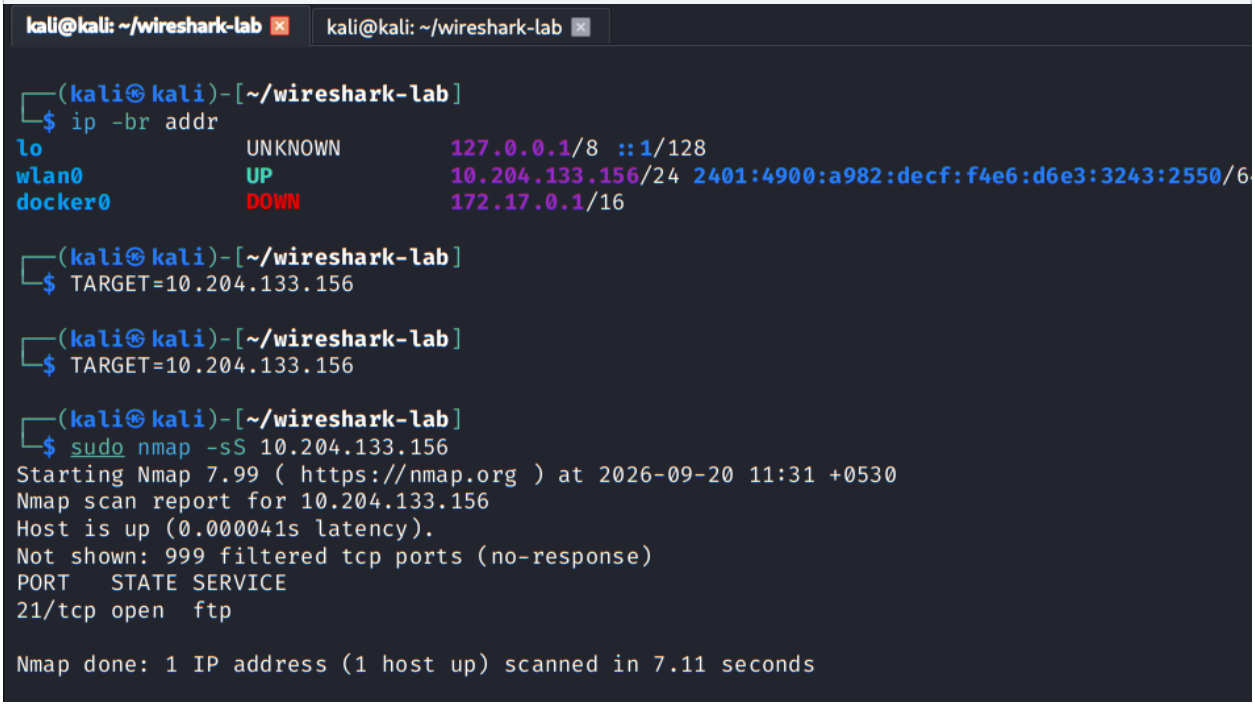


Figure 4.12: SYN scan of the laboratory host confirming TCP port 21 open with 999 ports filtered.

The scan reports a single open port with the remainder filtered. The distinction matters: a closed port answers with RST, whereas a filtered port answers not at all, so Nmap's classification is itself an inference drawn from packet-level behaviour of exactly the kind this report dissects. Running the same scan while capturing also produces the half-open signature discussed in Section 8.

7.2 Service and Session Setup

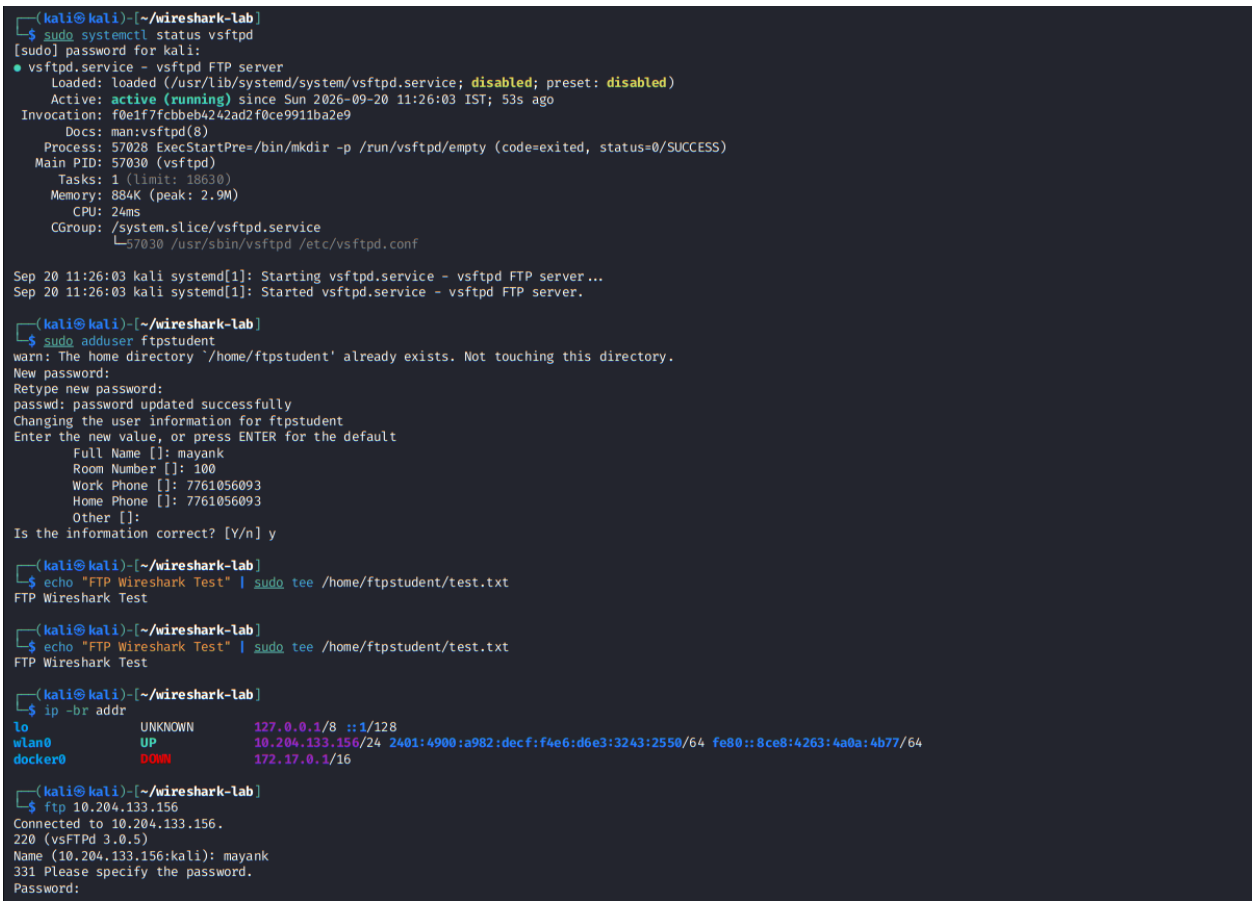


Figure 4.13: vsftpd 3.0.5 service status, test account creation and the start of an interactive FTP control session to port 21.

The vsftpd daemon was started, a test account was created, a small file was placed in its home directory, and an interactive session was opened to the service. The banner returned by the server, `220 (vsFTPd 3.0.5)`, is transmitted before any authentication takes place and discloses both the product and its exact version to anyone who connects or observes - the same information an attacker would use to select a known exploit. The server then issued `331 Please specify the password`, which confirms that the supplied username exists and thereby permits username enumeration.

7.3 The Cleartext Control Channel

Filtering the capture on `ftp` and following the TCP stream renders the control channel as a readable transcript, because FTP defines its commands as ASCII text terminated by carriage-return and line-feed. The USER and PASS commands appear in full.

```
220 (vsFTPd 3.0.5)
USER ftpstudent
331 Please specify the password.
PASS ******** <-- transmitted in cleartext on the wire
230 Login successful.
TYPE I
PASV
RETR test.txt
```

The password is shown masked in this report as a matter of documentation practice; in the capture itself it is present verbatim, exactly as typed. No cryptanalysis, no privileged position and no software vulnerability is required to obtain it - only the ability to observe frames on the segment, which on a shared wireless network is a low bar.

7.4 Active and Passive Data Channels

FTP separates control from data. In active mode the server opens a connection back to the client from port 20, which modern firewalls and NAT devices routinely block. In passive mode, negotiated by the PASV command, the server nominates a high port to which the client connects; the nominated address and port appear in the cleartext response and are therefore equally visible. Either way the transferred file itself is unencrypted, so `test.txt` and its contents are recoverable from the capture with Wireshark's export-objects facility.

7.5 The Exposure Chain

Stage	Source report	Finding
Discovery	Report I - Nmap	TCP port 21 open; service identified as FTP.
Assessment	Report II - Nessus	Cleartext authentication protocol flagged; banner disclosure noted.
Detection	Report III - Snort	Rules alerting on FTP login activity on the monitored segment.
Proof	Report IV - Wireshark	Credentials and file contents recovered directly from captured frames.

7.6 Remediation

- Replace FTP with SFTP over SSH, or enable FTPS so that the control channel is protected by TLS.
- If the service must remain, restrict it to a management network and require key-based access to that network.
- Suppress the version banner to reduce the value of the service to an opportunistic attacker.
- Disable anonymous access, enforce account-specific chroot, and log all authentication attempts.
- Retain the Snort signatures from the previous exercise so that any residual use of port 21 is alerted upon.

8. Attack Traffic Analysis and Anomaly Indicators

Protocol knowledge becomes security capability when it is applied to recognising traffic that deviates from the normal patterns established in the preceding sections. This section sets out the signatures an analyst should look for, grounded where possible in the evidence already captured.

8.1 Port Scan Signatures

The SYN scan in Section 7 produces a characteristic pattern when observed on the target. A single source address emits SYN frames to a large number of destination ports in rapid succession; open ports answer SYN-ACK and are then sent RST rather than the expected ACK, so no handshake ever completes; closed ports answer RST-ACK; filtered ports answer nothing at all, which is why the scan reported 999 ports in that state. The diagnostic filter is a high ratio of SYN frames to completed handshakes from one source.

```
Detection filter:
tcp.flags.syn == 1 && tcp.flags.ack == 0

Then use Statistics > Conversations to identify any single source
addressing an anomalous number of distinct destination ports.
```

8.2 Distinguishing Congestion from Attack

The retransmissions and duplicate acknowledgements observed in Section 3 are benign, and the reasoning matters more than the conclusion. They are distributed across several established conversations, they involve connections that completed their handshakes normally, they resolve without the connections being torn down, and they coincide with ICMP round-trip times above one second. A SYN flood presents the opposite profile: a very large number of half-open connections, typically from varied or spoofed source addresses, with no application data on any of them and no orderly termination.

Indicator	Benign congestion	Attack traffic
Source diversity	Few known peers	Many, often spoofed
Handshake completion	Normal	Never completes
Application data	Present after handshake	Absent
Timing	Irregular, bursty	Uniform, machine-paced
Destination ports	Few, service-appropriate	Many, enumerated
Resolution	Self-correcting	Persists until blocked

8.3 Other Indicators Relevant to This Capture

- **ARP and Neighbour Discovery anomalies** - the capture contains legitimate ICMPv6 Neighbour Solicitation and Advertisement exchanges. A spoofing attack manifests as repeated unsolicited advertisements binding a gateway address to an attacker's link-layer address; the filter `arp.duplicate-address-detected` and its IPv6 equivalent surface this directly.

- **DNS anomalies** - unusually long or high-entropy labels suggest tunnelling; responses arriving from an address other than the configured resolver, or bearing a transaction identifier that was never requested, suggest spoofing or cache poisoning.
- **ICMP anomalies** - echo requests at a rate far exceeding the one-per-second interval observed here, payloads that are not the standard incrementing pattern, or requests to a broadcast destination.
- **Cleartext credential exposure** - the FTP transcript in Section 7 is itself the signature. Equivalent exposure exists for Telnet, POP3, IMAP and SMTP without STARTTLS, and the same stream-following technique recovers it.
- **TLS anomalies** - negotiation of deprecated versions or weak cipher suites, self-signed certificates on public services, or an SNI value inconsistent with the certificate presented.

8.4 Analytical Workflow

The procedure applied throughout this report generalises into a repeatable method: begin with the capture summary and confirm zero drops; use Statistics > Protocol Hierarchy to characterise the trace as a whole; use Statistics > Conversations to find the heaviest or most unusual endpoint pairs; apply targeted display filters to isolate the protocol of interest; follow streams to reconstruct application-layer exchanges; inspect the details and bytes panes to verify field-level values; and finally correlate what has been found against external evidence such as scan results, vulnerability findings and intrusion-detection alerts. This last step is what converts packet analysis from a descriptive exercise into a security assessment.

9. Consolidated Findings

No.	Observation	Evidence	Assessment
1	Capture completed with zero dropped frames	Status bar, Fig. 4.1	Trace is reliable for loss and timing analysis
2	TCP handshake completes in approx. 21.5 ms	Frames 7-9, Fig. 4.4	Path round-trip time is acceptable for TCP
3	DNS queries and responses are in cleartext	Frames 3-6, Fig. 4.6	Browsing profile exposed to any on-path observer
4	TLS SNI discloses hostname before encryption	Frame 10 / 33, Fig. 4.9	Residual metadata exposure that TLS does not remove
5	HTTP exposes headers, body and any credentials	Section 5.1, Fig. 4.7	Unsuitable for any authenticated or sensitive use
6	25 percent ICMP loss with RTT above one second	Frames 34-54, Figs. 4.10-4.11	Degraded wireless path; loss occurs beyond the host
7	FTP service banner discloses vsftpd 3.0.5	Fig. 4.13	Enables targeted exploit selection
8	FTP credentials recoverable from the capture	Section 7.3	High severity; confirms Nessus finding with direct proof
9	TCP port 21 open, 999 ports filtered	Fig. 4.12	Attack surface minimal but the one service is the weak one
10	Retransmissions attributable to congestion	Fig. 4.5, Section 8.2	Benign; distinguished from attack by stated criteria

9.1 Recommendations

- Decommission plaintext FTP in favour of SFTP or FTPS, and treat any credential that has traversed the cleartext channel as compromised.
- Adopt an encrypted resolver protocol so that name resolution is not exposed on shared wireless segments.
- Restrict all authenticated services to TLS-protected transports and verify the negotiated version and cipher suite from the capture rather than from configuration alone.
- Investigate the wireless path quality, since the observed loss and latency will degrade every application running over it.
- Retain periodic baseline captures so that future traffic can be compared against a known-good profile, which is a precondition for reliable anomaly detection.

10. Conclusion

This experiment achieved each of its stated objectives. A live capture was taken without loss, and the three-pane interface was used to move from a frame summary to individual header fields and finally to the

raw bytes that encode them. A complete TCP three-way handshake was isolated with the SYN filter and shown, in frames 7 to 10, to be immediately followed by the TLS Client Hello - a direct demonstration of protocol layering in a real trace. DNS resolution was dissected as a pair of matched transactions and the returned records were tracked into the connections that used them. HTTP and HTTPS were compared side by side, establishing precisely which observables encryption removes and which it leaves intact. ICMP echo behaviour was analysed and a 25 percent loss rate reported by the tool was independently confirmed, and localised to a specific request, from the packet evidence.

The FTP section is the point at which the four reports in this series converge. Nmap discovered the open port, Nessus classified the weakness, Snort detected the activity, and Wireshark now supplies the irrefutable evidence: the credentials are present in the capture file in plain ASCII. The progression from discovery to proof is exactly the progression a competent security assessment follows, and the value of packet analysis is that it terminates the chain in fact rather than in inference.

The broader lesson concerns the limits of encryption. TLS decisively protects content and credentials, but the capture also shows the destination address, the hostname in both the DNS query and the TLS SNI extension, the timing of every exchange and the volume of data transferred. Security engineering must account for what remains visible as carefully as for what has been concealed.

11. References

[1] Postel, J. (1981). *Transmission Control Protocol*. RFC 793, Internet Engineering Task Force.

[2] Postel, J. (1981). *Internet Control Message Protocol*. RFC 792, Internet Engineering Task Force.

[3] Mockapetris, P. (1987). *Domain Names - Implementation and Specification*. RFC 1035.

[4] Postel, J. and Reynolds, J. (1985). *File Transfer Protocol*. RFC 959.

[5] Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446.

[6] Fielding, R. and Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*. RFC 7230.

[7] Sanders, C. (2017). *Practical Packet Analysis*, 3rd edition. No Starch Press.

[8] Wireshark Foundation. *Wireshark User's Guide* and *Display Filter Reference*.

[9] Lyon, G. (2009). *Nmap Network Scanning*. Insecure.Com LLC.

[10] Raj, M. (2026). *Lab Reports I-III: Nmap Port Scanning, Nessus Vulnerability Assessment and Snort Intrusion Detection*. MCA Network Security Laboratory.

Appendix A: Display Filter Reference

Filter	Purpose	Section
`tcp.flags.syn == 1`	All connection attempts and their responses	3
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	Initial SYN only - port scan detection	8
`tcp`	All TCP, including retransmission annotations	3
`tcp.analysis.retransmission`	Retransmitted segments only	3
`tcp.analysis.duplicate_ack`	Duplicate acknowledgements only	3
`dns`	DNS queries and responses	4
`dns.flags.response == 0`	Queries only	4
`http`	Cleartext HTTP traffic	5
`tls`	TLS records including handshake	5
`tls.handshake.type == 1`	Client Hello - reveals SNI	5
`icmp`	All ICMP messages	6
`ftp`	FTP control channel commands and replies	7
`ftp-data`	FTP data channel transfers	7
`ip.addr == 10.204.133.156`	All traffic to or from the laboratory host	All

Appendix B: List of Figures

Figure 4.1: Live unfiltered capture on wlan0 showing the packet list and packet details panes; 228 frames captured, 0 dropped. 6

Figure 4.2: Terminal session used to generate ICMP, DNS and HTTPS traffic during the capture window. 7

Figure 4.3: Display filter tcp.flags.syn == 1 applied; frame 7 selected, showing the initial SYN with Seq=0 and Len=0.9

Figure 4.4: Frames 7, 8 and 9 forming the complete three-way handshake, followed by the TLS Client Hello in frame 10. 10

Figure 4.5: Broader TCP view: 121 of 398 frames displayed, showing retransmissions, spurious retransmissions and duplicate acknowledgements. 11

Figure 4.6: DNS query and response pairs for example.com; frame 3 expanded to show the query structure and the highlighted name encoding in the bytes pane. 12

Figure 4.7: Python HTTP test server bound to port 8080 in the laboratory working directory. 14

Figure 4.8: curl request to example.com over HTTPS, returning the document body to the terminal. 14

Figure 4.9: Display filter tls applied; 28 of 74 frames displayed, showing Client Hello, Server Hello and Application Data records with TCP segment reassembly. 15

Figure 4.10: Output of ping -c 4 8.8.8.8 reporting three replies, 25 percent loss and round-trip times above one second. 17

Figure 4.11: Display filter icmp applied; seven frames displayed with frame 34 expanded to show the ICMP header, identifier, sequence number and payload. 17

Figure 4.12: SYN scan of the laboratory host confirming TCP port 21 open with 999 ports filtered. 19

Figure 4.13: vsftpd 3.0.5 service status, test account creation and the start of an interactive FTP control session to port 21. 20